# SET 18

## PART A — Short questions and answers

**Q1. Describe the role of an attribute domain, using Seats_Booked as an example.**

**Answer:** This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

**Q2. Interpret the statement: CREATE TABLE MOVIE (Movie_ID INT PRIMARY KEY, Title VARCHAR(50) NOT NULL);**

**Answer:** NOT NULL requires a column to have a value; it prevents NULL entries.

**Q3. Explain what makes a decomposition of the BOOKING relation lossless.**

**Answer:** This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

**Q4. Explain the difference between the AND and OR operators in a WHERE clause.**

**Answer:** This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

**Q5. Explain the purpose of the MAX function applied to Amount.**

**Answer:** This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

**Q6. Explain the use of RAISE_APPLICATION_ERROR.**

**Answer:** This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

**Q7. Compare relational and graph databases for storing recommendation data.**

**Answer:** A graph database stores entities as nodes and their connections as relationships.

**Q8. Describe the effect of the drop() method on a collection.**

**Answer:** This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

**Q9. Describe the output of the MongoDB query: db.movies.find({Genre: {$in: ["Drama", "Thriller"]}})**

**Answer:** The query returns all documents that satisfy the specified filter condition.

**Q10. Interpret the Cypher query: MATCH (u:User)-[r:RATED]->(m:Movie) WHERE r.stars >= 4 RETURN m.title;**

**Answer:** This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

## PART B — System solution

**Scenario:** A multiplex maintains a Movie Ticket Booking System to store user details, movie details, and ticket

**1(a) ER design:**

• **USER_ACCOUNT**(UserID, UserName, Email, City); primary key: **UserID**.

• **MOVIE**(MovieID, Title, Genre, Language, Duration); primary key: **MovieID**.

• **BOOKING**(BookingID, UserID, MovieID, ShowTime, SeatsBooked, Amount); primary key: **BookingID**.

Connect the two main entity tables through the transaction/relationship table. The transaction table has its own primary key and foreign keys to both entity tables. Cardinality: each entity can participate in many transactions; each transaction belongs to one row from each entity table (1:M on both sides, resolving M:N). Include the transaction's date/status/amount attributes shown in the question.

**1(b) SQL DDL/DML:** Create each table with its listed columns, PRIMARY KEY on its ID, and FOREIGN KEYs in the transaction table. Add NOT NULL to required fields; use CHECK/UNIQUE/DEFAULT where the question asks. Insert five sample rows per table, making sure foreign-key values exist.

```
CREATE TABLE USER_ACCOUNT (
  UserID NUMBER PRIMARY KEY,
  UserName VARCHAR2(100),
  Email VARCHAR2(100),
  City VARCHAR2(100)
);
CREATE TABLE MOVIE (
  MovieID NUMBER PRIMARY KEY,
```

```
  Title VARCHAR2(100),
  Genre VARCHAR2(100),
  Language VARCHAR2(100),
  Duration VARCHAR2(100)
);
CREATE TABLE BOOKING (
  BookingID NUMBER PRIMARY KEY,
  UserID NUMBER,
  MovieID NUMBER,
  ShowTime VARCHAR2(100),
  SeatsBooked VARCHAR2(100),
  Amount NUMBER
);
```

**2(a) Join query:** Select the two entity names/details from the transaction table by joining the transaction table to both entity tables using their IDs.

```
SELECT a.*, b.*
FROM BOOKING x
JOIN USER_ACCOUNT a ON x.UserID = a.UserID
JOIN MOVIE b ON x.MovieID = b.MovieID;
```

**2(b) Aggregate query:** Group the transaction rows by the requested category/type and apply HAVING to the aggregate condition stated in the question.

```
SELECT grouping_column, COUNT(*) AS total
FROM transaction_table
GROUP BY grouping_column
HAVING COUNT(*) > threshold;
```

**2(c) PL/SQL procedure:** Create a procedure with an IN parameter for the requested ID. Join the transaction table to the related entity tables, filter by that parameter, and print selected values using DBMS_OUTPUT.PUT_LINE.

```
CREATE OR REPLACE PROCEDURE show_details(p_id IN NUMBER) AS
BEGIN
  FOR r IN (
    SELECT a.*, b.*, x.*
    FROM transaction_table x
    JOIN entity_table_1 a ON x.entity1_id = a.entity1_id
    JOIN entity_table_2 b ON x.entity2_id = b.entity2_id
    WHERE x.transaction_id = p_id
  ) LOOP
    DBMS_OUTPUT.PUT_LINE('Details retrieved');
  END LOOP;
END;
/
```

**3. Graph database (Neo4j/Cypher):** Create the required node labels and relationship types exactly as specified in the set. Example pattern:

```
CREATE (a:EntityA {id: 1, name: 'Name A'});
CREATE (b:EntityB {id: 2, name: 'Name B'});
MATCH (a:EntityA {id: 1}), (b:EntityB {id: 2})
CREATE (a)-[:RELATED_TO]->(b);
MATCH (a:EntityA)-[:RELATED_TO]->(b:EntityB)
RETURN a.name, b.name;
```